PRACTICAL - 4

Implementation and Time Analysis of Factorial using Iterative and Recursive Methods

Aim of the Practical:

To implement factorial iteratively and recursively and compare their time and space complexities.

Program:

```
#include <iostream>
using namespace std;
long long factorialIterative(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact *= i;
    return fact;
}
long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}
int main() {
    int n = 5;
    cout << "Iterative: " << factorialIterative(n) << endl;
    cout << "Recursive: " << factorialRecursive(n) << endl;
}
```

Output:

```
Iterative: 120
Recursive: 120
```

Time Analysis:

Iterative Method - Derivation:

The loop starts at 2 and runs through n.

$$\text{Number of iterations} = n - 1$$

Each iteration performs constant work.

$$T_{iter}(n) = c(n - 1) + c_0 = \Theta(n)$$

Only a fixed number of variables are used, so auxiliary space is $\Theta(1)$.

Recursive method recurrence:

$$T_{rec}(n) = T_{rec}(n - 1) + c$$

Unrolling:

$$\begin{aligned} T_{rec}(n) &= T_{rec}(n - 2) + 2c \\ &= T_{rec}(1) + (n - 1)c \\ &= \Theta(n) \end{aligned}$$

There are n recursive stack frames in the worst case, so auxiliary space is $\Theta(n)$.

Final Time Analysis:

- Iterative Time: $\Theta(n)$, Space: $\Theta(1)$.
- Recursive Time: $\Theta(n)$, Space: $\Theta(n)$.

Result:

Both methods compute factorial correctly. Their time is linear, but recursion uses linear stack space.

Name: syed abdul kalam **Enrollment No: 92400118069**